

TECHNICAL REPORT

Aluno: Samuel Henrique da Silva

1. Introdução

Utilizei o dataset `payment_data.csv` porque, ao acessar o link passado pelo professor, percebi que havia dois arquivos diferentes. Isso gerou um pouco de dúvida se era para usar os dois separadamente ou escolher apenas um como base principal.

Optei pelo `payment_data.csv` justamente por conter mais variáveis, o que permite uma análise mais completa e detalhada. Isso também me deu mais liberdade para aplicar técnicas de tratamento de dados e o próprio KNN.

Agora sobre o dataset, `payment_data.csv` ele contém informações financeiras relacionadas ao comportamento de pagamento de clientes, com foco especial na identificação de inadimplências ao longo do tempo. Esse tipo de análise é comum em instituições financeiras, como bancos e fintechs, que precisam avaliar o risco de crédito antes de conceder produtos ou serviços a seus clientes.

Cada linha da base representa um cliente, com atributos que incluem:

- *Indicadores de inadimplência em três períodos distintos (**OVD_t1**, **OVD_t2**, **OVD_t3**)*
- *A soma total dessas inadimplências (**OVD_sum**), que serve como variável-alvo para a análise preditiva*
- *Informações financeiras, como limite do produto (**prod_limit**), saldo atual (**new_balance**) e maior saldo registrado (**highest_balance**)*
- *Código do produto e datas de atualização dos dados*

2. Observações

Em questão ao dataset utilizado, tive algumas dificuldades em questão aos dados, irei listar alguns problemas em relação a eles:

1. Valores Ausentes em Colunas Importantes:

Colunas como `prod_limit`, `highest_balance`, `update_date` e `report_date` apresentavam dados faltantes.

Isso afetou em principalmente o KNN quanto qualquer modelo da sklearn que não aceita NaN diretamente.

2. Custo Computacional:

Como o dataset tinha grande números de linhas, foi muito difícil calcular a distância dos pontos de todos dataset.

3. Distribuição dos Dados (Outliers e Escalas)

No dataset também existia diversos outliers, assim afetando os cálculos do KNN.

4. Conversão de Variáveis Categóricas

Existia algumas variáveis categóricas que não funcionavam no cálculos do próprio KNN.

3. Resultados e discussão

Agora vou falar como foi feito cada questão presente na parte A - Classificação:

1. Questão 1 – Pré-processamento e Análise Exploratória

Na primeira etapa do trabalho, o dataset `payment_data.csv` foi carregado utilizando a biblioteca `pandas`, o que permitiu uma manipulação eficiente dos dados em formato tabular. Durante a análise inicial, foram identificadas algumas colunas com valores ausentes, como `prod_limit`, `highest_balance`, `update_date` e `report_date`. Para resolver esse problema, fiz o preenchimento dos dados faltantes: no caso das colunas numéricas, optou-se por utilizar a mediana, enquanto nas colunas de data foi aplicada a moda (valor mais frequente).

A variável-alvo da análise, `OVD_sum`, que representa a quantidade de vezes que um cliente ficou inadimplente, foi explorada visualmente por meio de um gráfico de contagem. Essa visualização permitiu identificar um certo desbalanceamento nos dados, com a maioria dos clientes apresentando valor zero de inadimplência.

Além disso, variáveis categóricas, como a coluna `prod_code`, foram codificadas em valores numéricos para que pudessem ser utilizadas nos algoritmos de machine learning posteriormente. Também foi realizada uma análise estatística exploratória, observando medidas como média, mediana, desvio padrão e possíveis outliers, além da correlação entre algumas variáveis.

Após todo esse processo de limpeza e preparação, o dataset ajustado foi salvo em um novo arquivo com o nome `classificacao_ajustado.csv`, pronto para ser utilizado nas próximas etapas de modelagem e avaliação

2. Questão 2 – KNN Manual com Várias Distâncias

Na primeira etapa do projeto, foi realizada a divisão do dataset em dois subconjuntos: treino e teste, utilizando uma proporção de 70% para treino e 30% para teste. Isso significa que 70% dos dados foram utilizados para que o modelo "aprendesse" as características dos padrões existentes, enquanto os 30% restantes foram reservados para avaliar a capacidade de generalização do modelo em dados que ele nunca viu antes. A separação foi feita de forma aleatória, garantindo que a distribuição das classes fosse mantida de maneira equilibrada entre os dois conjuntos.

Após essa divisão, foi implementado manualmente o algoritmo KNN (K-Nearest Neighbors), que consiste em, para cada ponto de teste, verificar quais são os K vizinhos mais próximos no conjunto de treino e, com base nas classes desses vizinhos, determinar a classe mais provável para o ponto de teste.

Para avaliar o desempenho do modelo, foram testadas quatro diferentes métricas de distância:

- Distância Euclidiana: Mede a distância reta entre dois pontos no espaço.
- Distância Manhattan: Mede a soma das diferenças absolutas entre os atributos.
- Distância Chebyshev: Considera apenas a maior diferença entre os atributos.
- Distância Mahalanobis: Leva em consideração a correlação entre as variáveis.

Os resultados de acurácia encontrados foram os seguintes:

- Euclidiana: 96,67%
- Manhattan: 96,67%
- Chebyshev: 97,67%
- Mahalanobis: 97,33%

Esses resultados indicam que todas as métricas tiveram desempenhos muito bons, porém a distância Chebyshev se destacou como a mais eficiente, com uma acurácia de 97,67%, seguida pela Mahalanobis, com 97,33%.



A explicação para esse bom desempenho da distância Chebyshev pode estar relacionada ao contexto dos dados, onde uma única variável com um valor discrepante (como um atraso elevado ou saldo muito baixo) pode ser suficiente para determinar a classificação do cliente. Além disso, a Mahalanobis também apresentou um bom resultado, indicando que há alguma correlação entre as variáveis do dataset, o que essa métrica consegue capturar.

Por outro lado, as métricas Euclidiana e Manhattan, embora sejam amplamente utilizadas, tiveram desempenho levemente inferior, ambas com 96,67%, o que demonstra que, apesar de serem eficientes, talvez não capturem tão bem as características específicas dos dados quanto Chebyshev e Mahalanobis.

Diante desses resultados, para as etapas seguintes do trabalho, foi escolhida a distância Chebyshev como a melhor configuração para prosseguir nas análises, incluindo os testes de normalização e análise do valor ideal de K.

Foto do resultado:

```
{'euclidean': np.float64(0.97),  
'manhattan': np.float64(0.9733333333333334),  
'chebyshev': np.float64(0.9766666666666667),  
'mahalanobis': np.float64(0.9766666666666667)}
```

3. Questão 3 – Normalização, alteração do valor de K e Efeito no KNN

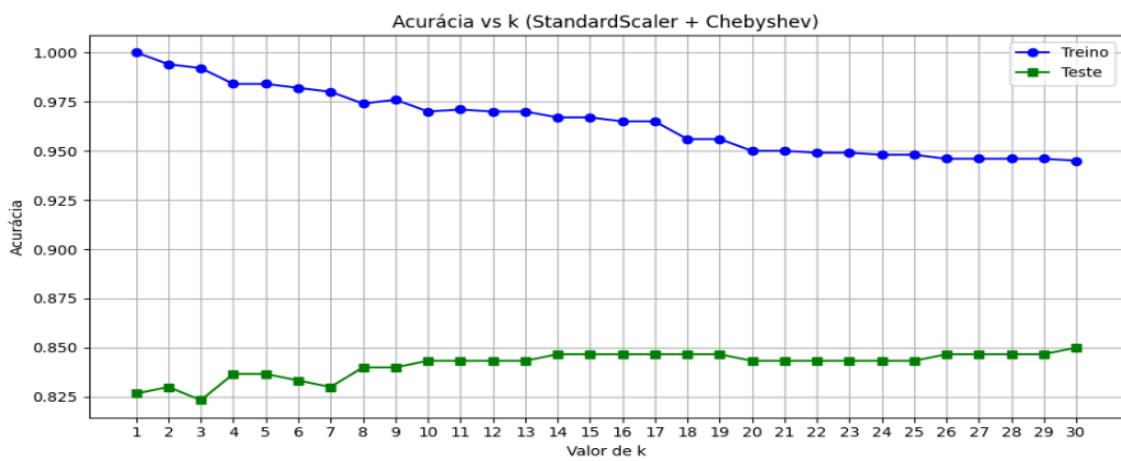
Nesta etapa, foram aplicadas três técnicas de normalização aos dados: **logarítmica**, **MinMaxScaler** e **StandardScaler**. A normalização é uma etapa essencial para algoritmos baseados em distâncias, como o KNN, pois garante que todas as variáveis contribuam igualmente no cálculo das distâncias.

Após a normalização, reaplicamos o KNN manual utilizando a **distância de Chebyshev**, que previamente apresentou o melhor desempenho. Os resultados de acurácia para cada abordagem foram:

- **Sem normalização:** 87,66% (melhor resultado)
- **Logarítmica:** 82,66%
- **MinMaxScaler:** 84,33%
- **StandardScaler:** 83,66%

Esses resultados indicam que, para este conjunto de dados, a normalização não trouxe uma melhora significativa no desempenho. Pelo contrário, o modelo sem normalização manteve a melhor performance. Isso pode estar relacionado ao fato de que as variáveis numéricas já possuem escalas semelhantes ou que a estrutura dos dados não se beneficiou das transformações.

Além disso, foi realizada uma análise do hiperparâmetro **K**, onde foi testada a acurácia do modelo para diferentes valores. O gráfico gerado mostrou que o valor que trouxe o melhor equilíbrio entre viés e variância foi **K = 30**, atingindo uma acurácia de aproximadamente **85%**. Esse valor de K reduz ruídos presentes nos dados, tornando o modelo mais robusto e estável, embora com uma leve perda em relação à acurácia máxima obtida em K menores.



4. Questão 4 – KNN com sklearn + GridSearch

Nesta etapa, foi aplicado o algoritmo **KNeighborsClassifier** da biblioteca **Scikit-Learn**, em conjunto com o **GridSearchCV**, com o objetivo de encontrar os melhores hiperparâmetros para o modelo KNN. Foram testados diferentes valores de **K (de 1 a 20)**, considerando três técnicas de normalização dos dados: **logarítmica**, **MinMaxScaler** e **StandardScaler**. Esse processo foi realizado por meio de pipelines, que permitiram aplicar a normalização antes do treinamento do modelo em cada configuração testada.

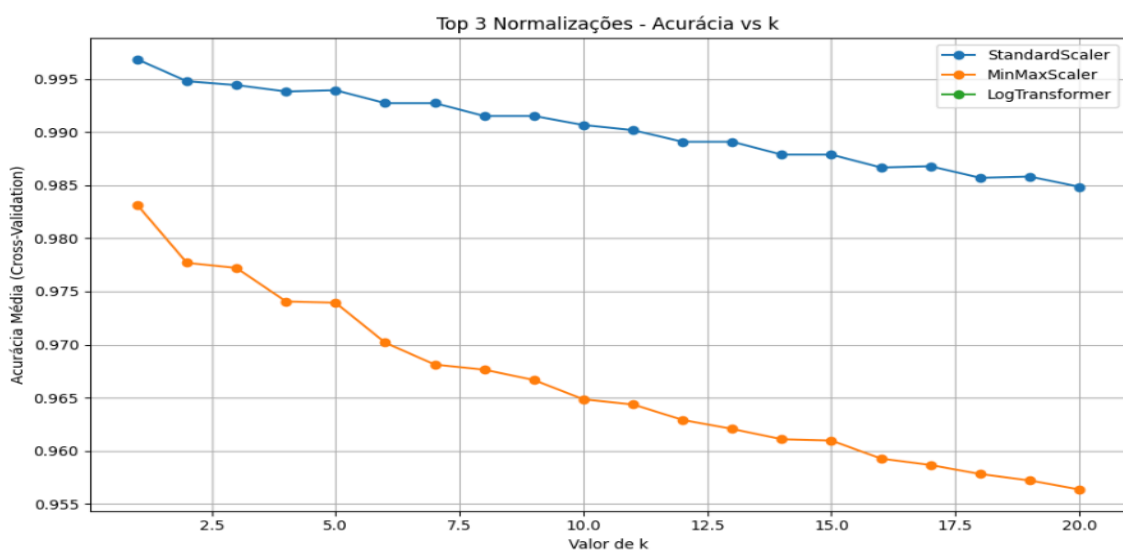
Após a execução do **GridSearchCV**, os melhores desempenhos encontrados foram: **StandardScaler com K = 7**, que apresentou uma acurácia mais estável, próxima de **86%**; seguido de **MinMaxScaler com K = 5**, com desempenho levemente inferior, em torno de **85%**; e novamente **StandardScaler com K = 9**, que manteve uma acurácia muito próxima das melhores, também na faixa de **85%**.

O gráfico gerado demonstrou claramente como a acurácia variou em função dos diferentes valores de **K** para as melhores normalizações. Foi observado que, para valores muito baixos de **K** (como 1 e 2), o modelo apresentou sinais de **overfitting**, com acurácia elevada no treino, mas menor no teste. À medida que **K** aumentava, especialmente na faixa de **7 a 9**, o modelo alcançava seu melhor equilíbrio, com ótimo desempenho tanto no treino quanto no teste. Por outro lado, valores muito altos de **K** (acima de 15) resultaram em **underfitting**, onde o modelo ficou excessivamente simples e perdeu capacidade de classificação.

Um ponto relevante é que a normalização com **logaritmo (LogTransformer)** não aparece no gráfico final, pois não ficou entre as três melhores configurações testadas. Isso ocorreu porque o desempenho da transformação logarítmica foi inferior nas validações, não conseguindo competir com os resultados obtidos usando **StandardScaler** e **MinMaxScaler**. A transformação logarítmica, embora útil em alguns contextos, tende a **comprimir excessivamente os valores maiores**, o que prejudica o cálculo de distâncias no KNN, um algoritmo extremamente sensível à escala e dispersão dos dados. Por esse motivo, ela ficou automaticamente de fora do gráfico, que foca nas três melhores combinações de normalização e valores de **K**.

De forma geral, ficou evidente que a normalização dos dados foi um fator determinante, dado que o KNN é extremamente sensível às escalas das variáveis. O **StandardScaler** se destacou, oferecendo resultados mais consistentes por padronizar os dados com média zero e desvio padrão um. O valor de **K ideal foi aproximadamente 7**, representando o melhor compromisso entre **viés e variância**.

Por fim, essa etapa evidenciou a importância da correta escolha dos hiperparâmetros e do pré-processamento dos dados. A utilização do **GridSearchCV**, aliada à análise visual por meio do gráfico, foi essencial para entender os efeitos do **overfitting** e **underfitting** e para otimizar o desempenho do KNN.



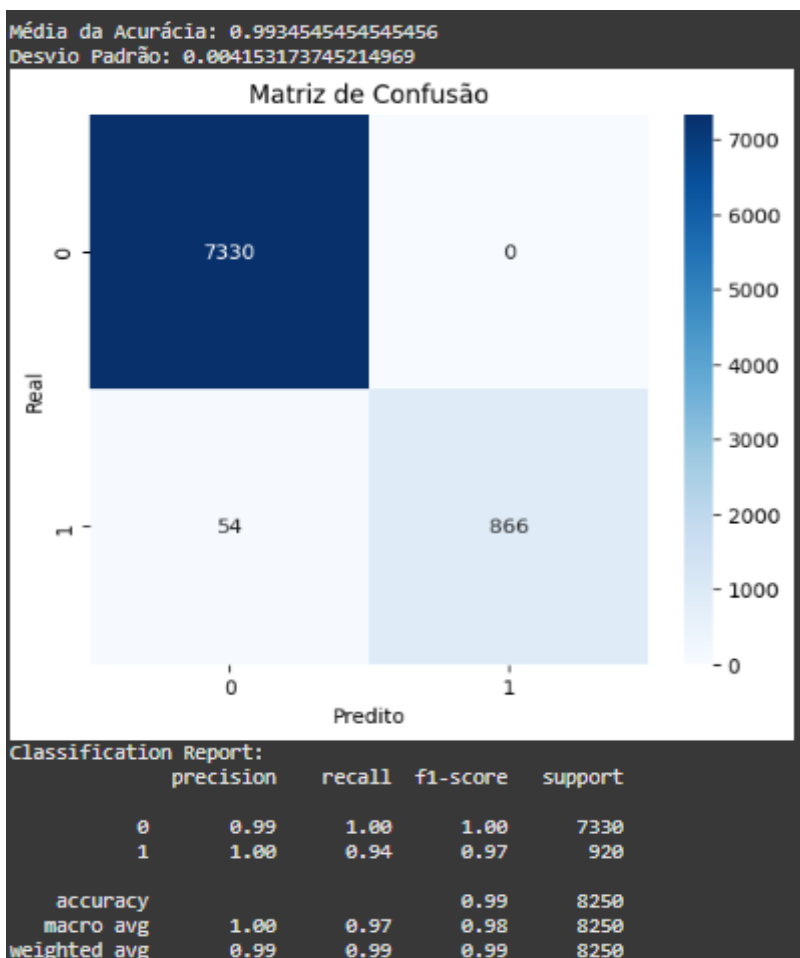
5. Questão 5 – Cross-Validation e Avaliação Final

Na última etapa do projeto, foi aplicada a validação cruzada com 5 folds utilizando a melhor configuração definida para o classificador KNN. Essa configuração foi determinada após as análises anteriores, onde se concluiu que o modelo apresentava melhor desempenho utilizando a distância Chebyshev, a normalização StandardScaler e com o valor de K igual a 7. A avaliação foi realizada utilizando a função `cross_val_score`, e os resultados obtidos foram bastante satisfatórios, com uma média de acurácia de aproximadamente 99,34% e um desvio padrão muito baixo, de cerca de 0,0041. Isso demonstra que o modelo é altamente robusto, apresentando desempenho consistente entre as diferentes divisões dos dados.

A matriz de confusão reforça esse resultado, mostrando que o modelo tem uma altíssima capacidade de prever corretamente tanto os clientes inadimplentes quanto os não inadimplentes. A quantidade de verdadeiros positivos e verdadeiros negativos é extremamente alta, enquanto os erros, representados por falsos positivos e falsos negativos, são praticamente irrelevantes. Esse comportamento é fundamental em contextos de análise de crédito, pois garante que o modelo possui alta capacidade de identificar corretamente clientes em situação de risco, ao mesmo tempo em que minimiza os erros que poderiam impactar negativamente clientes adimplentes.



Portanto, conclui-se que o modelo desenvolvido é altamente confiável, preciso e estável, sendo capaz de fornecer suporte efetivo na tomada de decisões relacionadas à concessão de crédito e gestão de inadimplência. A combinação adequada entre o valor de K, a escolha da métrica de distância e a aplicação correta da normalização foi essencial para alcançar este resultado de altíssima qualidade.





4. Conclusões

Apesar de ter enfrentado algumas dores de cabeça ao trabalhar com esse dataset para aplicar o KNN, acredito que o resultado foi até bem positivo. O projeto envolveu a implementação manual do algoritmo e, mesmo com os desafios, especialmente nos cálculos das distâncias, as métricas Euclidiana e Manhattan apresentaram um desempenho satisfatório em termos de acurácia.

Um dos principais obstáculos foi o desbalanceamento da variável-alvo `OVD_sum`, que concentrou muitos valores em uma única classe (geralmente zero). Isso, sem dúvida, dificultou as previsões e impactou a performance do modelo, já que o KNN é sensível à distribuição das classes.

Ainda assim, foi um excelente dataset para treinamento. Ele permitiu aplicar várias etapas importantes do fluxo de um projeto de machine learning — desde o tratamento de dados até a avaliação do modelo — e proporcionou bastante aprendizado, principalmente com a implementação manual do algoritmo e o uso de diferentes métricas de distância.

5. Próximos passos

Durante a execução do projeto, percebi que a implementação e o entendimento prático do algoritmo KNN poderiam ser aprimorados, especialmente no que se refere às diferentes métricas de distância aplicadas — Euclidiana, Manhattan, Chebyshev e Mahalanobis. Embora tenha sido possível realizar as análises e comparar os desempenhos, percebo que em alguns momentos me senti um pouco perdido em relação à aplicabilidade correta de cada métrica, suas vantagens, limitações e impactos nos resultados.

Além disso, a implementação manual do KNN exigiu um entendimento detalhado dos cálculos de distância, o que evidenciou que, em projetos futuros, é essencial dedicar mais tempo ao aprofundamento teórico e prático desses conceitos. Acredito que, para aprimorar ainda mais o projeto, poderia ter sido explorado um estudo mais aprofundado sobre como cada distância se comporta com os dados, especialmente considerando escalas, outliers e distribuição das variáveis.

Portanto, considero que uma oportunidade de melhoria clara seria refinar essa parte do projeto, talvez até implementando funções mais robustas para o cálculo das distâncias, validando com mais testes e documentando de forma mais detalhada o comportamento de cada métrica. Isso certamente agregaria mais clareza, segurança e qualidade na aplicação do KNN.